

Inbetriebnahme der Funkstrecke

Stand nach der Übernahme der Endfassung. Reihenfolge nach §12 der Arbeitsanweisung, an den tatsächlichen Aufbau angepasst.

Ganz eilig? Die Checkliste zum Abhaken steht am Ende des Dokuments.

Was bereits erledigt ist

Damit du nicht selbst nachsehen musst — am Code und an der Projektmappe ist nichts mehr zu tun:

- Alle neuen Dateien liegen in den echten Projektordnern, nicht mehr in `chatty` oder `claudine`.
- `RadioProtocol.h` ist in `1_Common.vcxitems` eingetragen, `Nrf24Radio` in `2_Hardware.vcxitems`, `RadioTelemetrySender` und `TelemetryOutput` in `5_System.vcxitems` — jeweils auch in den zugehörigen `.filters`.
- Das Empfängerprojekt `funk_empfaenger` ist angelegt und in `robot.sln` eingetragen.
- `FrontApp` erzeugt und benutzt die Funkobjekte, `TelemetryPrinter` und `CommandRunner` geben über `TelemetryOutput` aus.
- Alle drei Ziele übersetzen fehlerfrei.

Offen ist nur, was ich nicht kann: bauen in Visual Micro, flashen, und der Test an der Hardware.

0. Vorher

Neu bauen ist Pflicht. Ich habe Quelldateien geändert; die vorhandenen Hex-Dateien sind veraltet. `flash_beide.ps1` merkt das und bricht ab — der Schutz ist gewollt, nicht umgehen.

In Visual Studio:

1. Projektmappe neu laden. `funk_empfaenger` ist neu dazugekommen.
2. Beim Projekt `funk_empfaenger` Board auf **Nano, Old Bootloader** stellen und den COM-Port des dritten Nano wählen.
3. `vorne`, `hinten` und `funk_empfaenger` in Release neu kompilieren.

Erwartete Größen, falls Visual Micro etwas anderes meldet, bitte sagen:

	Flash	SRAM
vorne	26422 von 30720 (86 %)	1306 von 2048 (63 %)
hinten	13368 (43 %)	706 (34 %)
funk_empfaenger	4338 (14 %)	396 (19 %)

Der hintere Nano ist bitgenau wie vorher. Wenn er sich verändert hat, stimmt etwas nicht — dann melden, bevor geflasht wird.

1. Verdrahtung

Sender und Empfänger haben unterschiedliche CE- und CSN-Pins. Das ist Absicht: der Sender liegt laut Arbeitsanweisung fest auf D7 und D8, der Empfänger behält die schon aufgebaute Verdrahtung aus dem ersten Testsketch. SPI ist bei beiden gleich.

Signal	Nano vorne (Sender)	Empfänger-Nano
CE	D7	D9
CSN	D8	D10
MOSI	D11	D11
MISO	D12	D12
SCK	D13	D13
VCC	3,3 V , nicht 5 V	3,3 V
GND	GND	GND

Zwei Punkte, beide **Vermutung als Fehlerquelle**, nicht beobachtet:

- Das nRF24-Modul zieht kurze Stromspitzen, die der 3,3-V-Regler des Nano schlecht abfängt. Ein Kondensator von 10 bis 100 µF direkt zwischen VCC und GND am Modul ist der übliche Abhilfe. Wenn der Funk unerklärlich abreißt, ist das der erste Verdächtige.
- An D13 hängt zusätzlich die Bord-LED als Last am SPI-Takt. Zweiter Verdächtiger, falls die Übertragung unzuverlässig bleibt.

2. Empfänger allein

Nur den Empfänger flashen und den seriellen Monitor mit **115200 Baud** öffnen.

Erwartet:

```
#RADIO, ready
#RSTAT,0,0,0,0,0
```

Danach alle fünf Sekunden eine **#RSTAT**-Zeile mit Nullen.

Steht dort **#RADIO,error,init**, hat **radio.begin()** das Modul nicht gefunden. Dann Verdrahtung und 3,3 V prüfen, nicht die Software.

Felder von **#RSTAT**:

```
#RSTAT, empfangen, vollstaendig, verworfen, Duplikate, unvollstaendig
```

3. Sender dazu

Vorderen Nano flashen, hinteren wie gehabt. Am **USB-Monitor des vorderen Nano** muss unmittelbar nach dem Start stehen:

```
#INFO, Radio, ok
```

Steht dort `#INFO, Radio, fehlt`, ist das Funkmodul am vorderen Nano nicht erkannt worden. Wichtig: der Roboter läuft trotzdem normal weiter, die Telemetrie über USB bleibt vollständig. Es fehlt nur der Funk.

4. Erste Übertragung im Stand

Ohne Fahrbefehl entstehen kaum Telemetriezeilen. Was in jedem Fall kommt, ist alle fünf Sekunden die Statuszeile vorne:

```
#RTX, gesendeteZeilen, verworfene, Pakete, Fehlpakete, Ringhoechststand, maxSendUs
```

Diese Zeile geht selbst über Funk. Sie muss also **auf beiden Monitoren** auftauchen: am vorderen Nano über USB und am Empfänger über Funk. Das ist der einfachste Vollstreckentest.

Was die Felder sagen:

Feld	gut	schlecht
verworfene Zeilen	bleibt 0 oder wächst sehr langsam	wächst stetig mit
Fehlpakete	einzelne	in der Größenordnung der gesendeten
Ringhöchststand	deutlich unter 240	nahe 240
maxSendUs	unter 3000	dauerhaft am Anschlag

5. Mit Fahrbefehl, aufgebockt

Roboter aufbocken, damit die Räder frei laufen. Fahrbefehl starten.

Am Empfänger müssen jetzt `#WHEELS` und `#ODOM` im Frametakt erscheinen, also etwa alle 80 ms. Vergleich mit dem USB-Monitor des vorderen Nano: dort stehen dieselben Zeilen. Sie müssen inhaltlich übereinstimmen.

Worauf ich achten würde:

- **Fehlen ganze Zeilentypen?** Wenn `#ODOM` per Funk nie ankommt, wohl aber über USB, wäre genau der Fehler zurück, wegen dem der Ringpuffer gebaut wurde.
- **Kommen verstümmelte Zeilen an?** Das darf nicht passieren. Bei Verlust fällt eine ganze Zeile weg, nie ein Teil. Wenn doch, bitte die Zeile wörtlich schicken.
- `maxSendUs`. Das ist der Wert, für den der Zähler eingebaut wurde. Er entscheidet, ob `setRetries(1, 2)` bleiben kann.

Was die Blockierung wirklich betrifft

`RF24::write()` ist synchron. Der Aufruf kehrt erst zurück, wenn das ACK da ist oder die Wiederholungen aufgebraucht sind. Gerechnet sind das etwa 0,6 ms im Normalfall und bis zu 2,8 ms, wenn zwei Wiederholungen nötig werden.

Unbeeinflusst, weil interruptgesteuert:

- Encoder-Zählung über PCINT — es geht kein Tick verloren
- Serial-Empfang, 64-Byte-Puffer; 2,8 ms entsprechen rund 32 Zeichen
- Ultraschall-Echomessung über PCINT1
- `millis()` und `micros()`

Beeinflusst ist allein der Zeitpunkt, zu dem der PI-Regler rechnet. Das ist unkritisch, weil `Rad::update()` die tatsächlich verstrichene Zeit misst und an den Regler weitergibt:

```
const uint16_t dt_ms = (uint16_t)(nowMs - _lastUpdateMs); // Rad.cpp
_integral += _Ki * e * dt_s;                               // PIRegler.cpp
```

Ein verspäteter Zyklus rechnet also mit 22,8 ms statt 20 ms, nicht mit einem falschen dt. Der Integralanteil wird nicht verfälscht.

Zum Vergleich: `Serial.print` blockiert bereits heute, sobald der 64 Byte grosse Sendepuffer voll ist. Bei einer 102 Zeichen langen `#WHEELS`-Zeile sind das rund 3,3 ms — dieselbe Größenordnung, seit jeher vorhanden.

Falls `maxSendUs` am Fahrzeug doch zu hoch liegt, ist die Notbremse ein Einzeiler in `Nrf24Radio::begin()`:

```
_radio.setAutoAck(false);
```

Dann kehrt `write()` nach etwa 0,35 ms zurück, konstant und ohne Wiederholungen. Der Preis: `failedPackets` sagt danach nichts mehr aus, weil es keine Bestätigung mehr gibt.

6. Langsame Fahrt

Erst danach, und erst wenn Schritt 5 sauber war. Hier interessiert vor allem, ob die Regelung unruhiger wird als vorher.

Wichtig für die Beurteilung: ohne eine Aufzeichnung derselben Fahrt vor dem Funkeinbau lässt sich das nicht entscheiden. Wenn du eine alte Messung derselben Strecke hast, sag Bescheid — sonst wäre eine Vergleichsfahrt mit dem vorherigen Firmwarestand der saubere Weg.

7. Python

Der Empfänger gibt dieselben Zeilen aus wie der vordere Nano über USB. `robot_python` kann später auf den COM-Port des Empfängers gelegt werden. `#RSTAT` und `#RTX` kennt der Parser nicht; er gibt für unbekannte Zeilen `False` zurück und ignoriert sie. Das stört also nicht.

Was ich von dir brauche, wenn etwas klemmt

Aus der Arbeitsregel im Projekt:

- Steht der Roboter aufgebockt oder auf dem Boden?

- Welcher Firmwarestand liegt auf welchem der drei Nanos? Nach dem Umbau ist das die häufigste Fehlerquelle.
- Sind Abstands- oder Zeitangaben gemessen oder geschätzt?
- Trat der Effekt einmalig auf oder ist er reproduzierbar?

Am hilfreichsten sind ein paar wörtliche Zeilen von beiden Monitoren gleichzeitig, samt der **#RTX**- und **#RSTAT**-Zeile aus demselben Zeitraum.

Checkliste

Zum Abhaken. Wenn ein Punkt nicht stimmt, nicht weitermachen — die Erklärungen stehen oben im jeweiligen Abschnitt.

Vorbereiten

- ☐ Projektmappe in Visual Studio neu geladen, **funk_empfaenger** ist sichtbar
- ☐ **funk_empfaenger**: Board **Nano, Old Bootloader**, COM-Port gesetzt
- ☐ **vorne** neu kompiliert → rund **26422** Flash, **1306** SRAM
- ☐ **hinten** neu kompiliert → **13368** Flash, **706** SRAM, unverändert
- ☐ **funk_empfaenger** kompiliert → **4338** Flash, **396** SRAM

Stopp, wenn **hinten** sich verändert hat.

Verdrahtung prüfen

- ☐ Sender: CE **D7**, CSN **D8**, MOSI D11, MISO D12, SCK D13
- ☐ Empfänger: CE **D9**, CSN **D10**, MOSI D11, MISO D12, SCK D13
- ☐ beide Module an **3,3 V**, nicht an 5 V
- ☐ GND beider Module verbunden

Empfänger allein

- ☐ Empfänger geflasht, Monitor mit **115200** offen
- ☐ **#RADIO, ready** erscheint
- ☐ alle 5 s eine **#RSTAT**-Zeile mit Nullen

Stopp bei **#RADIO, error, init** — Verdrahtung und 3,3 V prüfen.

Sender dazu

- ☐ vorderen und hinteren Nano geflasht
- ☐ am USB-Monitor vorne steht **#INFO, Radio, ok**
- ☐ Roboter startet normal, UART zum hinteren Nano läuft

Bei **#INFO, Radio, fehlt** fährt der Roboter weiter, nur ohne Funk.

Vollstreckentest im Stand

- ☐ #RTX erscheint alle 5 s am **USB-Monitor vorne**
- ☐ dieselbe #RTX-Zeile erscheint am **Empfänger**
- ☐ verworfene Zeilen bleiben 0 oder wachsen sehr langsam
- ☐ Ringhöchststand deutlich unter 240
- ☐ maxSendUs unter 3000

Mit Fahrbefehl, aufgebockt

- ☐ Roboter aufgebockt, Räder laufen frei
- ☐ #WHEELS und #ODOM kommen im Frametakt am Empfänger an
- ☐ beide Zeilentypen kommen an, nicht nur einer
- ☐ keine verstümmelten Zeilen am Empfänger
- ☐ Motorregelung läuft ruhig wie vorher
- ☐ maxSendUs und Zählerstände notiert

Stopp, wenn ein Zeilentyp systematisch fehlt oder Zeilen verstümmelt ankommen. Beides bitte wörtlich schicken.

Langsame Fahrt

- ☐ Schritt davor war fehlerfrei
- ☐ Vergleichsaufzeichnung von vor dem Funkeinbau vorhanden oder bewusst darauf verzichtet
- ☐ langsame Fahrt gefahren
- ☐ Regelung unauffällig
- ☐ #RTX und #RSTAT am Ende notiert

Danach

- ☐ Flash- und SRAM-Verbrauch aus Visual Micro notiert
- ☐ Zählerstände ausgewertet
- ☐ Ergebnis an mich, mit Angabe: aufgebockt oder auf dem Boden, welcher Firmwarestand auf welchem Nano, gemessen oder geschätzt, einmalig oder reproduzierbar